Лекция

23.10.2020

Адрес переменной

Каждая переменная любого типа в С связана с сопоставленным её блоком в оперативной памяти. Длина блока изменяется в байтах, для разных типов - различна. **Адресом** переменной называют **номер первого байта** из блока, который отведён под неё. В языке С это целое положительное число.

У переменной можно узнать адрес с помощью оператора - **&**.

Вывести значение адреса переменной можно с помощью спецификатора **%p** функции **printf**.

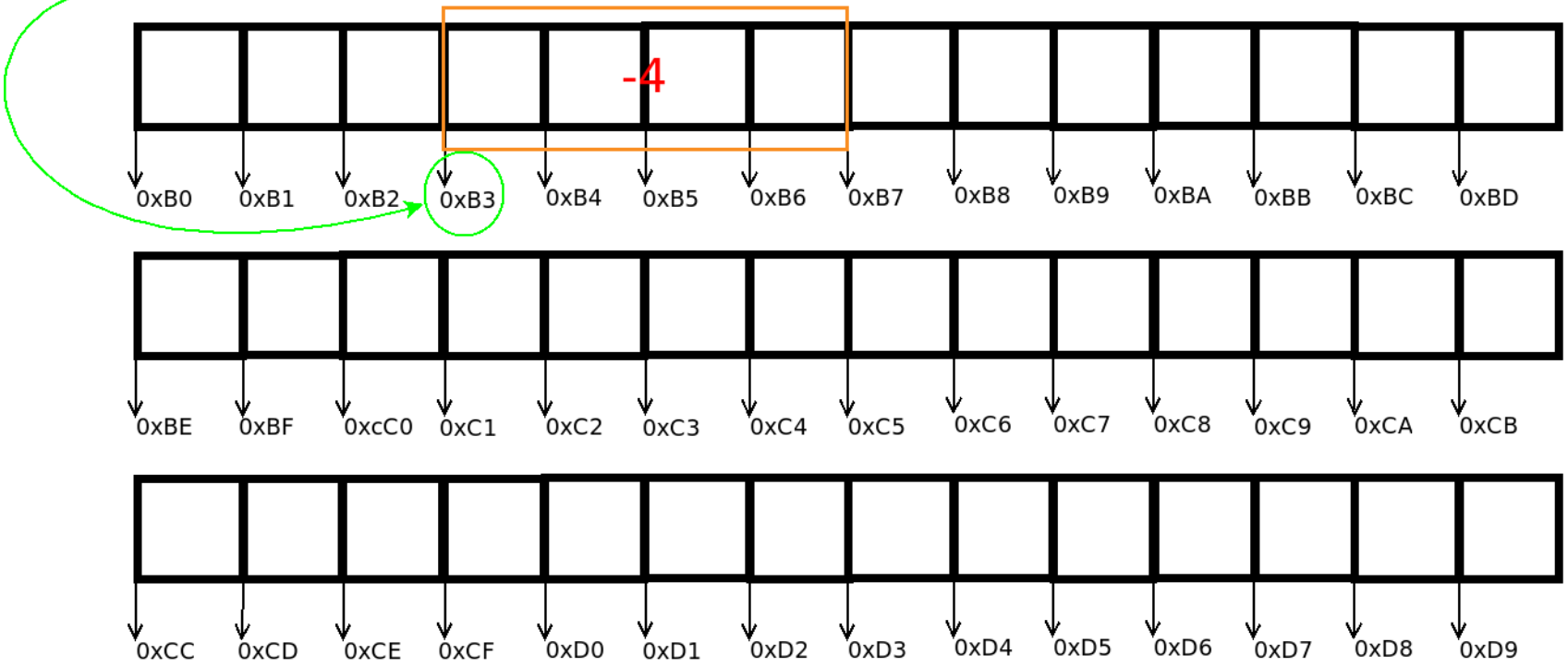
```
1 int scale = 5;
2 double rate = 3.4;
3
4 printf("Адрес rate: %p\n", &rate);
5 printf("Адрес scale: %p\n", &scale);
```

Значения адреса выводятся на экран в виде шестнадцатиричных чисел.

Адрес переменной

Как адрес выглядит графически: переменная **scale** имеет адрес равный **0xB3** и состоит из четырёх байт.

int scale = -4;
&scale;



Указатели

Указателем - называют тип данных, переменные которого предназначены для хранения адресов других объектов (то есть, обыкновенных переменных простых, специальных или пользовательских типов). Указатели в C являются типизированными.

Синтаксис объявления указателя

```
<тип_данных> *<имя_переменной>;
```

```
1 int    *p_int;    // указатель на int
2 char   *p_char;   // указатель на char
3 double *p_double; // указатель на double
```

Для практического применения указатели должны быть типизированными, чтобы было возможно получать значения тех переменных, адрес которых хранится в них.

Операции с указателями: **присвоение значения**
Указателю может быть присвоено значение (адрес в памяти) либо с помощью операции взятия адреса у переменной, либо копированием значения из другого указателя.

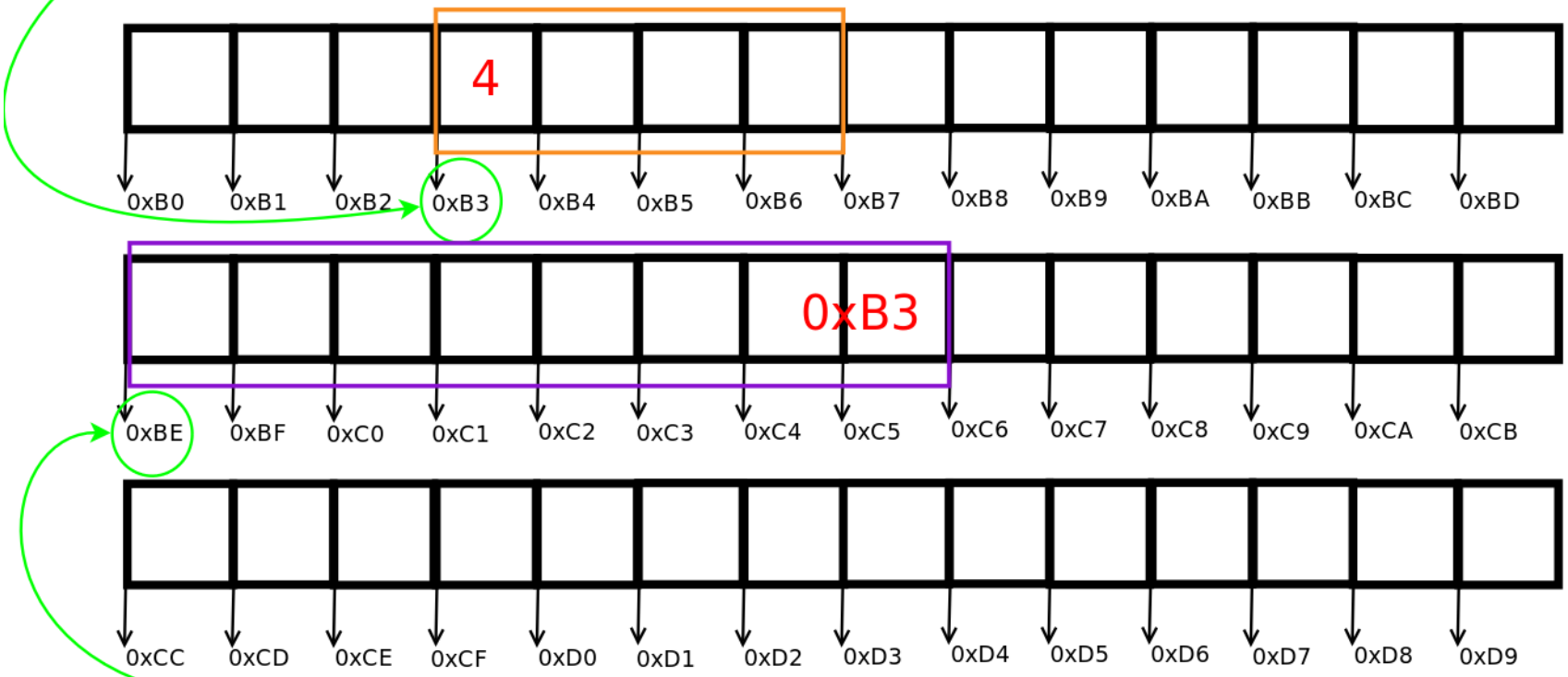
```
1 int scale = 5, *p_sc;
2
3 p_sc = &scale;
4 int *p2 = p_sc;
```

1-я строка: определяем переменную целого типа **scale** и указатель на целое **p_sc**.
3-я строка: получаем адрес **scale** и записываем его значение в переменную-указатель **p_sc**.
4-я строка: копируем значение (целое число) из **p_sc** в переменную-указатель **p2**.

Указатели

В памяти картина следующая. Обратите внимание, сама по себе **p_sc** - просто переменная, со своим адресом.

```
int scale = 4;
&scale;
```




```
int *p_sc = &scale;
printf("Адрес scale: %p\n", p_sc);
printf("Адрес p_sc: %p\n", &p_sc);
```

Указатели

Операции с указателями: **присвоение значения**.
Существует специальное значение для указателей, которое означает, что сама переменная-указатель не содержит реальный адрес какого-либо байта. Для обозначения такого спец. значения в C применяется специальная константа **NULL**. Также эта константа известна как **нулевой адрес**. Сам указатель с таким значением иногда называется как **нулевой указатель**.

```
1 double *p1 = NULL;
2 ...
3
4 if (p1 != NULL) {
5   // работаем с указателем p1
6 }
```

Правило для работы с указателями: переменная-указатель **всегда** должна быть определена, а не объявлена. То есть, ей надо или присвоить реальный адрес, или значение **NULL**.

Указатели

Операции с указателями: **разыменование** - получение значения переменной, адрес которой сохранён в указателе.

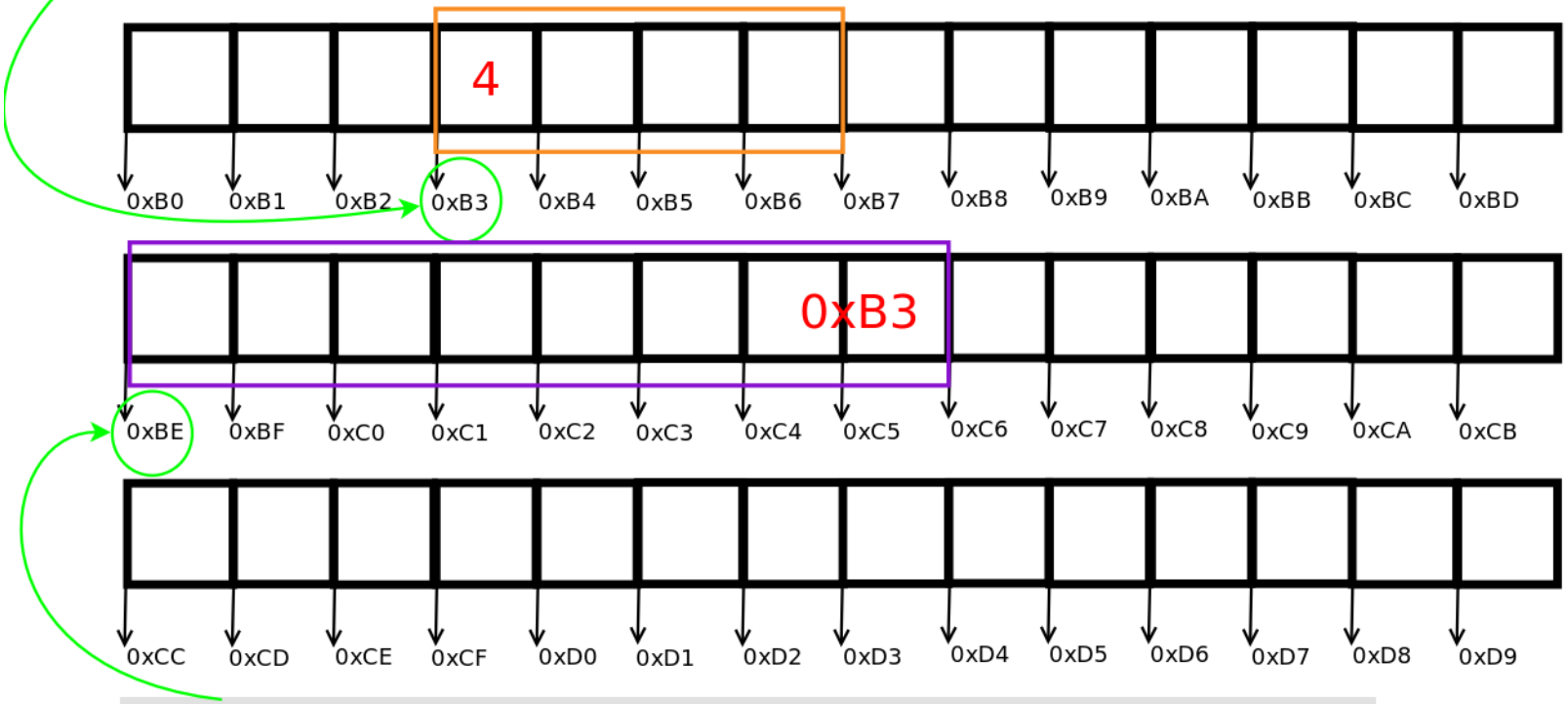
Синтаксис

*<имя_переменной>;

```
1  int scale = 5;
2  int *p_sc = &scale;
3
4  // Вывод 5 на экран
5  printf("p_sc ссылается на значение: %d\n", *p_sc;
6
7  // При разыменовании указатель может
8  // участвовать в арифметических выражениях
9  int rate = (*p_sc) + 15;
10
11 // изменяем значение scale через указатель p_sc
12 *p_sc = 15;
```

Схематично разыменование можно показать так:

```
int scale = 4;
&scale;
```




```
int *p_sc = &scale;
printf("Значение scale через указатель: %d\n", *p_sc);
*p_sc = 15;
printf("А теперь: %d", *p_sc);
```

Операции с указателями: **разыменование**

Предупреждение: разыменование объявленной, но не определённой, переменной-указателя **чрезвычайно опасно** в коде на C. Тоже самое верно и для указателей, которым присвоен **NULL**.

Так никогда не делать!

```
1 char *p_ch;
2 printf("Что же за символ у нас: %c", *p_ch);
3
4 int *p_int = NULL;
5 printf("Может с int получится: %d", *p_int);
```

Правило для работы с указателями: никогда не разыменовывать объявленный или **нулевой** указатель.

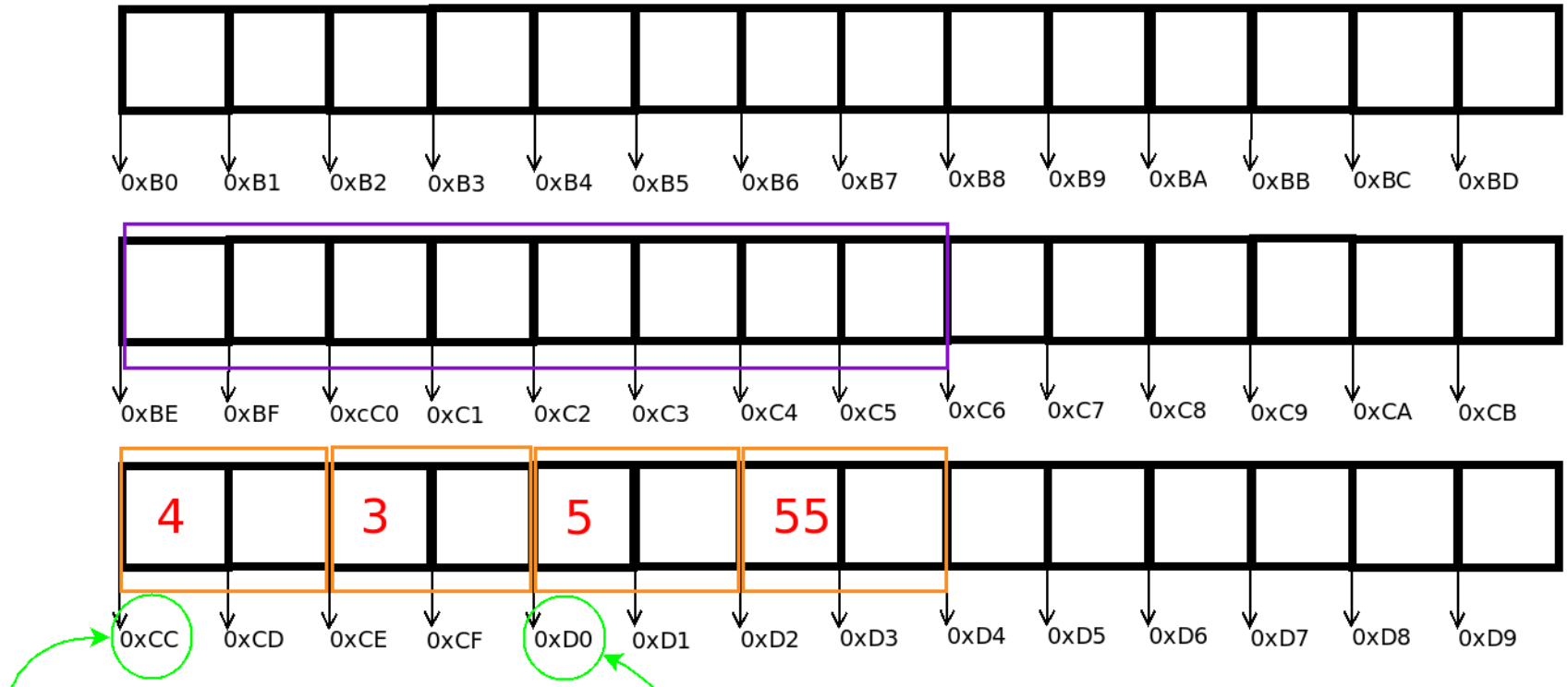
Операции с указателями: **разыменование**

Указатели дают возможность *менять* значения переменных, которые передаются в функции. Как пример:

```
1  void my_swap(int *i1, int *i2)
2  { // Обмен значениями двух переменных
3    if (i1 != NULL && i2 != NULL) {
4      int tmp = *i1;
5      *i1 = *i2;
6      *i2 = tmp;
7    }
8  }
9
10 int n1 = 11, n2 = -15;
11 my_swap(&n1, &n2);
12 printf("n1 = %d, n2 = %d\n", n1, n2);
```

Стоит отметить, что сами переменные-указатели передаются в функции **по-значению**. Здесь отличий от переменных обычных типов нет.

Вспомним, как массив располагается в памяти




```
short vec[4] = {4, 3, 5, 55};

&vec[0];

&vec[2];
```

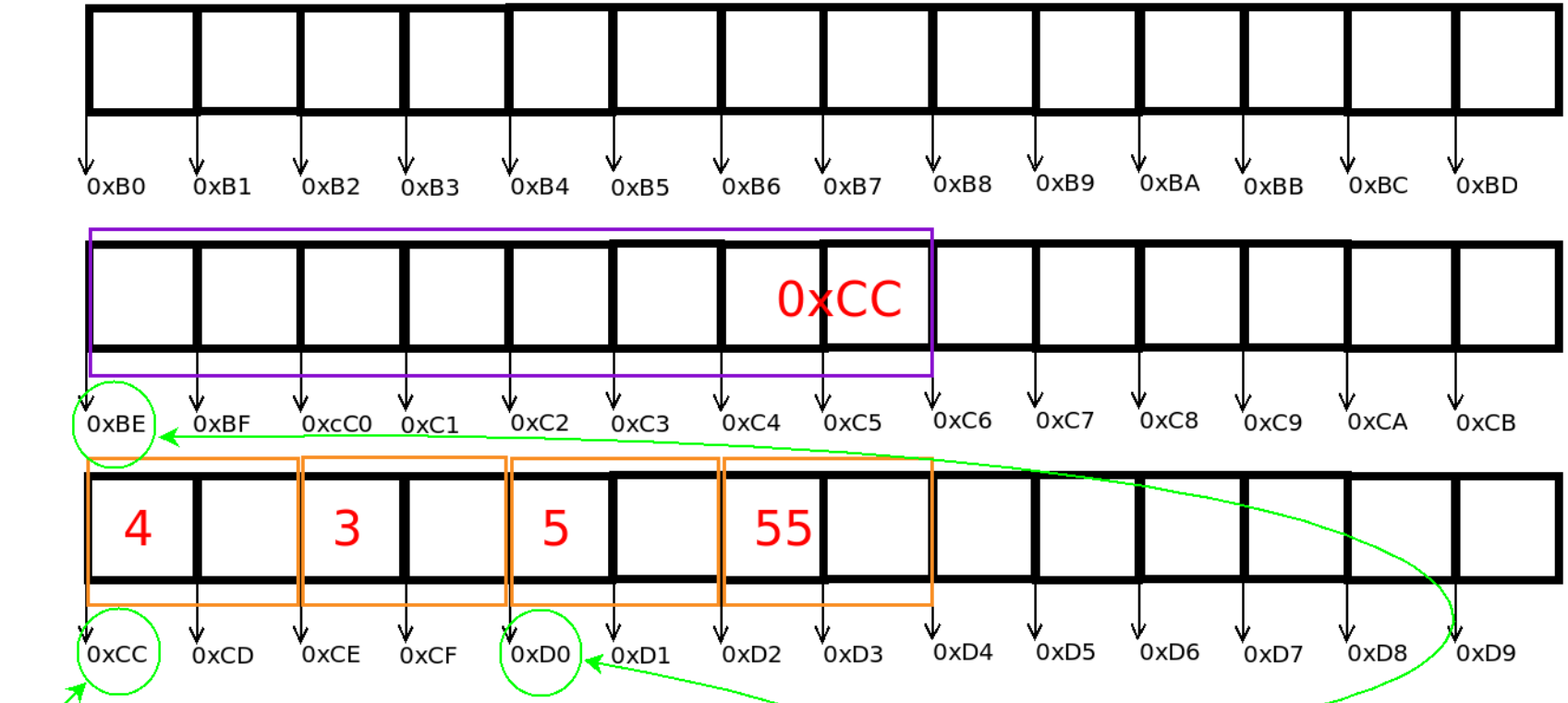
Указатели и массивы

Связь указателей и массивов

- Имя переменной-массива (выше - **vec**) является указателем на его первый элемент
- Массивы передаются в функцию как указатели. Поэтому, в любую функцию, принимающую указатель, можно передать переменную-массива. И наоборот
- Но: переменной массива **нельзя** присвоить никакой другой адрес (в отличии от переменной-указателя)

```
1 void print_array(short* arr, size_t count);
2 ...
3 short vec[4] = {4, 3, 5, 55};
4
5 print_array(vec, 4);
```

Картина в памяти: в указатель записали адрес массива




```
short vec[4] = {4, 3, 5, 55};

&vec[0];
&vec[2];
```

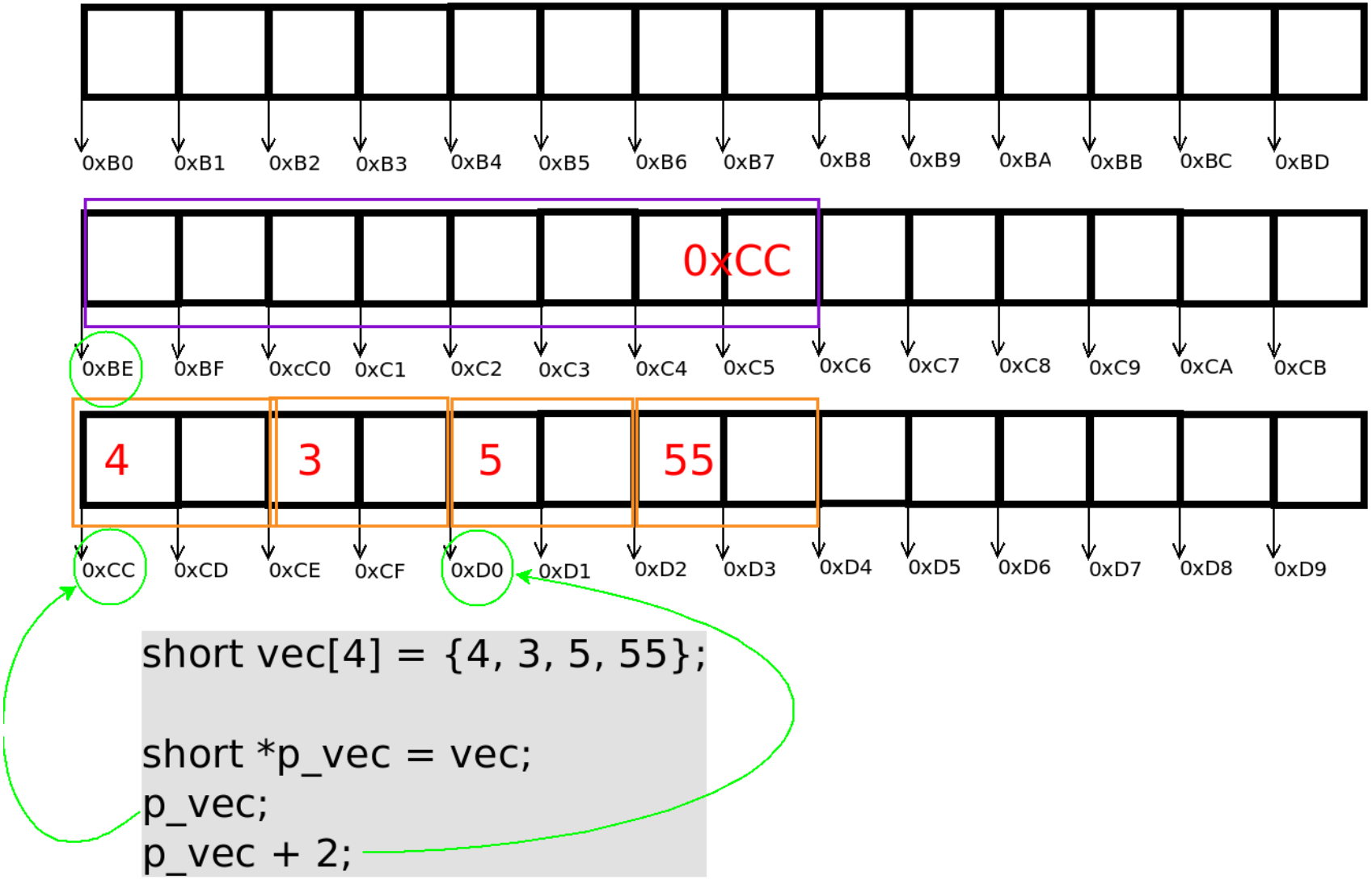
```
short *p_vec = vec;

printf("Число: %d", *p_vec);
```

Операции с указателями: **сложение с целым числом**
Результатом операции прибавления целого числа **n** к указателю является новый указатель, значение (адрес) которого смещено на $\mathrm{n} * \mathrm{sizeof}(< \mathrm{type} >)$ байт (вправо или влево - зависит от знака **n**).

```
1 short vec[4] = {4, 3, 5, 55};
2 short p_vec = vec;
3
4 // Перемещаемся на третий элемент
5 printf("Значение третьего элемента: %d\n",
6         *(p_vec + 2));
```

Смещение происходит блоками, размер которого определяется типом указателя (указатель на **int**, **double**, **char** и прочие).

Сложение указателя с целым числом на схеме с памятью




```
short vec[4] = {4, 3, 5, 55};

short *p_vec = vec;
p_vec;
p_vec + 2;
```

Операции с указателями: **сложение с целым числом**
Как видно из примеров, особенно полезно сложение при использовании указателя для работы с элементами массива.

```
1 short vec[4] = {4, 3, 5, 55};
2 short *p_vec = vec;
3
4 // Инкремент, как сложение
5 // с единицей, также допустим
6 p_vec++;
7 p_vec += 1;
8 p_vec -= 2;
```

Указатели и массивы

Операции с указателями: **индексация**

Индексация указателя выполняет два действия:

1. Сместиться на количество блоков, равных индексы, от текущего адреса
2. Получить значение по адресу, **после смещения**

```
1 short vec[4] = {4, 3, 5, 55};
2 short *p_vec = vec;
3
4 if (p_vec[2] == *(p_vec + 2)) {
5   puts("Значения равны");
6 }
7
8 printf("4-ый элемент: %d", vec[3]);
```

Указатели и массивы

Операции с указателями: **вычитание однотипных указателей**
Результатом вычитания является целое число (как положительное, так и отрицательное), показывающее количество блоков памяти между двумя адресами. Под блоком памяти, напоминаем, понимается размер типа указателя.

```
1  short vec[4] = {4, 3, 5, 55};
2  short *p1 = vec, *p2 = &vec[3];
3
4  int diff = p2 - p1;
5  // Печатает 3
6  printf("От начала массива до последнего "
7         "элемента поместились ещё %d эл-ов\n",
8         diff);
9
10 diff = p1 - p2;
11 // Печатает -3
12 printf("А так: %d\n",  diff);
```

Указатели

В отличие от переменных, можно объявлять **указатель на тип void**.

1. Указателю на **void** может быть присвоено значение любого другого указателя
2. Указатель на **void** может быть присвоен любой другой указатель только с использованием **явного приведения типа** (про приведение - два слайда ниже).
3. Для указателей на **void** запрещены операции **разыменования**, **индексации** и **вычитания указателей**.

```
1 short vec[4] = {4, 3, 5, 55};
2 short *p1 = vec, *p2;
3
4 void *pv1 = p1;
5 p2 = (short *) pv1;
```

Явное приведение значений разных типов в C

Неявные приведения

В арифметических операторах целое значение приводится к действительному типу тогда и только тогда, когда вещественным является один из операндов.

```
1  int i1 = 5, i2 = 9;
2  double d1 = 2.0;
3
4  double result = (i2 / i1) * d1;
5  printf("Результат (1): %lf\n", result);
6  // Будет выведено 2.0
7
8  result = (i2 / d1) * i1;
9  printf("Результат (2): %lf\n", result);
10 // Будет выведено 22.5
11
12 int tot_part = result;
13 printf("Целая часть равна: %d", tot_part);
14 // Будет выведено 22
```

Явные приведения

Значения могут быть приведены к другому типу явно. Общий синтаксис:

```
( <тип> ) <значение>;
```

Пример с численными типами

```
1 double rate = 34.505;
2 int scale = 6;
3
4 int tot_rate = (double) rate;
5 double real_scale = (int) scale;
```

Явные приведения

Более полезный случай использования явного приведения - преобразование указателей

```
1  double rate = 34.505;
2  int scale = 6;
3
4  double *d_pointer = &rate;
5  int *i_pointer = &scale;
6
7  char *b_pointer;
8
9  b_pointer = (char*) d_pointer;
10 printf("Первый байт rate равен %d", *b_pointer);
11
12 b_pointer = (char*) i_pointer;
13 printf("Первый байт scale равен %d", *b_pointer);
```

Динамическое выделение памяти

Динамическое выделение памяти

Существует необходимость в переменных, время жизни которых мы можем контролировать самостоятельно. Кроме того, нам необходимо динамическое выделение памяти, когда размер используемого пространства заранее не известен. Для этого используется выделение памяти на куче. Недостатков у такого подхода два: во-первых, память необходимо вручную очищать, во-вторых, выдеение памяти – достаточно дорогостоящая операция.
Для выделения памяти на куче в си используется функция **malloc** (memory allocation) из библиотеки **stdlib.h**

```
void * malloc(size_t size);
```

Функция выделяет size байтов памяти и возвращает указатель на неё. Если память выделить не удалось, то функция возвращает NULL.

Динамическое выделение памяти

Так как malloc возвращает указатель типа void, то его необходимо явно приводить к нужному нам типу. Например, создадим указатель, после этого выделим память размером в 100 байт.

```
1  #include <conio.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  void main() {
6      int *p = NULL;
7      p = (int*) malloc(100);
8
9      free(p);
10 }
```

После того, как мы поработали с памятью, необходимо освободить память функцией free.

Динамическое выделение памяти

Пример: вводим размер массива, создаём массив этого размера, заполняем его квадратами чисел и удаляем массив.

```
1 #include <conio.h> #include <stdio.h>
2 #include <stdlib.h>
3 void main() {
4  const int maxNumber = 100;
5  int *p = NULL;
6  unsigned i, size;
7  do {
8   printf("Enter number <%d: ", maxNumber);
9   scanf("%d", &size);
10     } while (size > maxNumber);
11   p = (int*) malloc(size * sizeof(int));
12     for (i = 0; i < size; i++) {
13         p[i] = i*i;printf("%d ", p[i]);  }
14     free(p); }
```

Динамическое выделение памяти

Здесь (int *) – приведение типов. Пишем такой же тип, как и у указателя. size * sizeof(int) – сколько байт выделить. sizeof(int) – размер одного элемента массива.

```
p = (int*) malloc(size * sizeof(int));
```

Функция malloc выделила память на куче по определённому адресу, после чего вернула его. Теперь указатель p хранит этот адрес и может им пользоваться для работы. Когда функция malloc «выделяет память», то она резервирует место на куче и возвращает адрес этого участка. У нас будет гарантия, что компьютер не отдаст нашу память кому-то ещё. Когда мы вызываем функцию free, то мы освобождаем память, то есть эта память может быть использована чем-то другим. Сама переменная не зануляется, она продолжает хранить адрес, которым ранее пользовалась. Функция free освобождает память, но не изменяет значение указателя, о чём нужно помнить(лучше присвоить NULL).

динамический двумерный массив

Для динамического создания двумерного массива сначала необходимо создать массив указателей, после чего каждому из элементов этого массива присвоить адрес нового массива.

Динамическое создание двумерного массива: выделение памяти

```
1 #include <conio.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 void main() {
5 float **p = NULL;
6 unsigned i,COL_NUM,ROW_NUM;
7 printf("cols: ");
8 scanf("%d", &COL_NUM);
9 printf("rows: ");
10 scanf("%d", &ROW_NUM);
11 p=(float**) malloc(ROW_NUM*sizeof(float*));
12 for (i = 0; i < ROW_NUM; i++) {
13 p[i]=(float*)malloc(COL_NUM*sizeof(float));
14     } // продолжение на след. слайде
```

Динамическое создание двумерного массива: освобождение памяти

Для удаления массива необходимо повторить операцию в обратном порядке - удалить сначала подмассивы, а потом и сам массив указателей.

```
1  // ввод элементов
2      for (i = 0; i < ROW_NUM; i++) {
3       for (j = 0; j <COL_NUM; j++) {
4           scanf("%f",&p[i][j]);
5           }
6      }
7  // Освобождение памяти
8      for (i = 0; i < ROW_NUM; i++) {
9           free(p[i]);
10     }
11     free(p);
12 }
```

Динамическое создание двумерного массива

Сначала мы создаём массив указателей, а после этого каждому элементу этого массива присваиваем адрес вновь созданного массива. Это значит, что можно

1. Создавать массивы "неправильной формы то есть массив строк, каждая из которых имеет свой размер.
2. Работать по отдельности с каждой строкой массива: освобождать память или изменять размер строки.

Создадим "треугольный"массив и заполним его значениями

```
1 #define SIZE 10
2 void main() {
3 int **A;
4 int i, j;
5 A = (int**) malloc(SIZE * sizeof(int*));
6 for (i = 0; i < SIZE; i++) {
7 A[i]=(int*) malloc((i + 1)*sizeof(int));}
8     for (i = 0; i < SIZE; i++) {
9         for (j = i; j > 0; j--) {
10            A[i][j] = i * j;
11        printf("%d ", A[i][j]);}
12    printf("\n"); }
13    for (i = SIZE-1; i > 0; i--) {
14        free(A[i]);  }
15    free(A); }
```

Выделение памяти

Функция **calloc** возвращает указатель на выделенную память. Размер выделенной памяти равен величине num*size, где size задается в байтах. Функция calloc() выделяет достаточно памяти для массива из num объектов каждый размером size байт и **заполняет их нулями**.

```
void* calloc(size_t num, size_t size);
```

Выделение памяти

Функция calloc() возвращает указатель на первый байт выделенной области. Если памяти недостаточно для удовлетворения запроса, то возвращается нулевой указатель. Всегда важно проверять возвращаемое значение на его равенство NULL, прежде чем использовать этот указатель. Следующая функция возвращает указатель на динамически выделенный массив для 100 чисел типа float:

```
1  float *get_mem(void)
2  {
3  float *p;
4  p = (float *) calloc(100, sizeof(float));
5  if(!p) {
6  printf ("Allocation failure.");
7  exit (1);
8  }
9  return p;
10 }
```

Выделение памяти

Ещё одна важная функция – **realloc** (re-allocation). Она позволяет изменить размер ранее выделенной памяти и получает в качестве аргументов старый указатель и новый размер памяти в байтах:

```
void* realloc(void* ptr, size_t size)
```

Функция realloc может как использовать ранее выделенный участок памяти, так и новый. При этом не важно, меньше или больше новый размер – менеджер памяти сам решает, где выделять память.

Создание массива с неизвестным заранее размером

```
1  void main() {
2      float *nums = NULL;
3      float readNext;
4      unsigned maxSize = 10, curSize = 0;
5      char next[2];
6      unsigned i, j;
7  //Выделили под  maxSize элементов
8  nums=(float*)malloc(maxSize*sizeof(float));
9      do {
10   //Пока вводим строку, начинающуюся с y(Y)
11 printf("next? [y/n]: ");
12 scanf("%1s", next);
13 if (next[0] == 'y' || next[0] == 'Y') {
14 scanf("%f", &readNext);
```

Создание массива с неизвестным заранее размером

```
1 //если количество элементов превысило ↩
    заданное
2 if (curSize >= maxSize) {
3 maxSize += 1;
4 // выделяем дополнительную память
5 nums=(float*)realloc(nums,maxSize*sizeof(↩
    float));
6 }
7 nums[curSize] = readNext;
8 curSize++;
9 } else {
10 break;
11 }
12 } while(1);
13 }
```

Ошибки при освобождении памяти

В выделенной области хранятся данные об объекте - его размер. При удалении free получает эту информацию. Однако, если мы изменили указатель, то удаление приведёт к ошибке, например

```
1 void main() {
2     int *p = NULL;
3 if (!(p = (int*) malloc(100 * sizeof(int))))
4  { printf("Error"); exit(1);     }
5     //Изменили указатель
6 p++;  //Теперь free не может найти ↩
    метаданные об объекте
7 free(p);//На некоторых компиляторах ошибки ↩
    не будет
8 }
```

Если указатель хранит адрес, то его не нужно изменять, лучше создать другой указатель, с которым работать дальше.

Ошибки при освобождении памяти

Вы удаляете объект, но при этом забываете изменить значение указателя на NULL. В итоге, он хранит адрес области памяти, которой уже нельзя воспользоваться, но проверить это нет возможности.

```
1  #define SIZE 10
2  void main() {
3      int *p = NULL;
4      p = (int*) malloc(SIZE * sizeof(int));
5      for (int i = 0; i < SIZE; i++)
6       { p[i] = i; }
7      free(p);
8      for (int i = 0; i < SIZE; i++)
9       { printf("%i ", p[i]); }
10 }
```

Эта программа отработает и выведет мусор, или не мусор, или не выведет. Поведение не определено.

Ошибки при освобождении памяти

Если же мы напишем

```
1 free(p);
2 p = NULL;
```

то программа выкинет исключение. Это определённо лучше, чем неопределённое поведение. Если вы освобождаете память и используете указатель в дальнейшем, то обязательно обнулите его.

Это помогает и при повторном освобождении памяти.

```
1     int *a, *b;
2     a = (int*) malloc(sizeof(int));
3     free(a);
4     a = NULL;
5     b = (int*) malloc(sizeof(int));
6     free(a);//Вызов free(NULL) ничего не ↩
        делает
```